

UNIT: 6**Java Framework**

The Java programming language was originally created in 1995 by James Gosling from Sun Microsystems (currently a subsidiary of Oracle Corporation). The goal was to provide a simpler and platform-independent alternative to C++.

Java is a general-purpose programming language that's used in all industries for almost any type of application.

Java is a platform-independent language. It's because when you write Java code, it's ultimately written for JVM but not your physical machine (computer).

6.1 Java Virtual Machine (JVM):

- JVM (Java Virtual Machine) is an abstract machine that enables the computer to run a Java program.
- When we run the Java program, Java compiler first compiles the Java code to bytecode. Then, the JVM translates bytecode into native machine code (set of instructions that a computer's CPU executes directly).
- The Java Virtual Machine provides a platform-independent way of executing code; programmers can concentrate on writing software, without having to be concerned with how or where it will run. But, note that JVM itself not a platform independent.
- It only helps Java to be executed on the platform-independent way.
- Different JVM is designed for different OS and byte code is able to run on different OS. So it is platform dependent whereas java is not.

Life Cycle of a Java Program

- Java requires the source code of our program to be compiled first.
- It gets converted to either machine-specific code or a bytecode that is understood by some run-time engine or a virtual machine.
- Not only will the program be checked for syntax errors by a Java compiler, but also some other libraries of Java code can be added (linked) to our program after the compilation is complete (deployment stage).
- Technically we can write the source code of our Java program in any plain text editor that we prefer (Notepad, Text Edit, vi, etc.), but to compile our program we'll need additional tools and code libraries that are included in the Java Development Kit (JDK).

**6.2 Java Runtime Environment (JRE):**

JRE (Java Runtime Environment) is a software package that provides Java class libraries, along with Java Virtual Machine (JVM), and other components to run applications written in Java programming. JRE is the superset of JVM. Java Runtime Environment (JRE) contains JVM, class libraries, and other supporting files. It does not contain any development tools such as compiler, debugger, etc. Actually JVM runs the program, and it uses the class libraries, and other supporting files provided in JRE. If we want to run any java program, we need to have JRE installed in the system.

6.3 Java Development Kit (JDK):

JDK (Java Development Kit) is a software development kit to develop applications in Java. Java Developer Kit contains tools needed to develop the Java programs, and JRE to run the programs. The tools include compiler (javac.exe), Java application launcher (java.exe), Applet viewer, etc. Compiler converts java code into byte code. Java application launcher opens a JRE, loads the class, and invokes its main method. We need JDK, if at all we want to write our own programs, and to compile the program. For running java programs, JRE is sufficient. JRE is targeted for execution of Java files i.e. JRE = JVM + Java Packages Classes (like util, math, lang, awt, swing etc.)+runtime libraries. JDK is mainly targeted for java development. i.e. we can create a Java file (with the help of Java packages), compile a Java file and run a java file.

❖ JIT(Just in Time Compiler):

The Just-In-Time (JIT) compiler is a component of the Java Runtime Environment that improves the performance of Java applications at run time. Nothing in the JVM affects performance more than the compiler, and choosing a compiler is one of the first decisions made when running a Java application whether you are a Java developer or an end-user. Initially Java has been accused of poor performance because it's both compiles and interprets instruction. Since compilation of Java file to class file is independent of execution of Java program. Here compilation word is used for byte code to machine instruction translation. JIT are advanced part of Java Virtual machine which optimize byte code to machine instruction conversion part by compiling similar byte codes at same time and thus reducing overall execution time. JIT is part of Java Virtual Machine and also performs several other optimizations such as in-lining function.

❖ Bytecode:

Bytecode is object-oriented programming (OOP) code compiled to run on a virtual machine (VM) instead of a central processing unit (CPU). The VM transforms program code into readable machine language for the CPU because platforms utilize different code interpretation techniques. A VM converts bytecode for platform interoperability, but bytecode is not platform-specific. Bytecode is in a compiled Java programming language format and has the .class extension executed by Java Virtual Machine (JVM).

Important Questions from this Unit for PU exams:

Q.1) Differences between JDK, JRE and JVM

Q.2) write short notes on either JDK, JRE or JVM